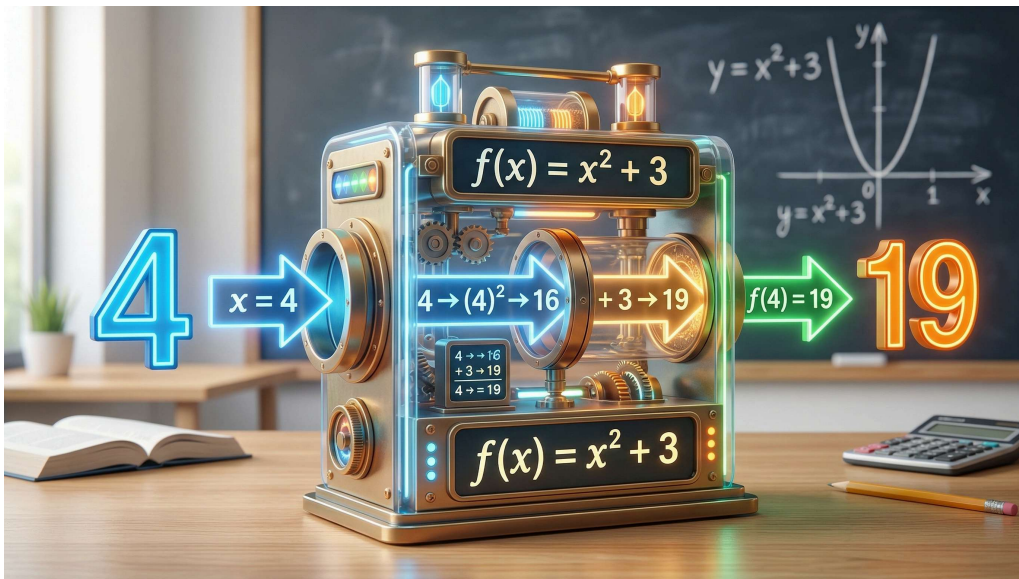


11. Funciones

Estás harto de ver expresiones matemáticas como esta: $f(x) = x^2 + 3$. Sabes perfectamente cómo funciona: le das un valor a la x (por ejemplo, 2), la función hace su cálculo ($2^2 + 3$), y te devuelve un resultado (7). En R, una función es exactamente lo mismo, pero a lo bestia. Es una “máquina” a la que tú le das unos ingredientes (datos o parámetros), la máquina sigue unas instrucciones que tú has escrito dentro, y te escupe un resultado final. La ventaja es que en R no solo le puedes meter números; le puedes meter textos, vectores, o tablas enteras.



11.1. Sintaxis general **function**

```
nombre_funcion <- function(argumentos)
{
  operaciones;
  return(resultado)
}
```

11.2. Crear funciones en R

Crear funciones propias en R es muy sencillo.

Ejemplo 1:

Imagina que quieres sumar repetidamente dos números (parámetros).

```
# Definir una función: suma de dos

mifuncion <- function(a,b)
{
  resultado <- a+b    # Las líneas entre corchetes van metidas con cuatro espacios hacia la derecha
  return(resultado)
}

# Llamar a la función
mifuncion(5,2)
```

```
## [1] 7
```

Explicación:

Paso 1: La creación: `mifuncion <- function(a,b)`

- Aquí le estamos dando un nombre a nuestra máquina, que llamamos `mifuncion`. Puedes llamar a tu función como quieras.
- Al poner `function(a, b)`, le estamos diciendo a R: “Esta máquina/función necesita obligatoriamente dos datos de entrada (ingredientes) para funcionar”. A estos datos los llamamos **argumentos**, y los hemos bautizado como `a` y `b`. Puedes llamarles como quieras, por ejemplo, `x` e `y`.

Paso 2: El motor: `{ resultado <- a + b }`

- Las llaves `{ }` delimitan el “motor” de la función, es decir, todas las **instrucciones** que la máquina ejecutará en orden.
- `resultado <- a + b` es la operación interna. R toma los valores que le pasaste, los suma y guarda el resultado en una variable temporal llamada `resultado`.
- Las instrucciones tienen que ser indentadas hacia la derecha (cuatro espacios en blanco a su izquierda).

Paso 3. La entrega: `return(resultado)`

- Esta es la parte final. Sin el `return`, la función haría el cálculo pero lo guardaría internamente sin mostrártelo. El `return` es como la bandeja de salida que te entrega el valor calculado al terminar el

proceso.

Paso 4. La ejecución: `mifuncion(5, 2)`

- Es el momento de la verdad. Aquí estás **invocando/llamando** a la función para que se ponga a funcionar.
 - Toma el **5** y lo asigna a la `a`.
 - Toma el **2** y lo asigna a la `b`.
- Ejecuta el código interno ($5 + 2$) y te devuelve un **7** en la consola.

Ejemplo 2:

Imagina que quieres reutilizar varias veces el código que eleva al cuadrado un número que va a cambiar.

```
# Creamos La función
funcioncuadrado <- function (a)
{resultado <- a^2
return(resultado)
}
```

```
# Llamamos a nuestra función
funcioncuadrado(6)
```

```
## [1] 36
```

```
#Podemos llamarla Las veces que queramos
funcioncuadrado(4)
```

```
## [1] 16
```

```
funcioncuadrado(3)
```

```
## [1] 9
```